
Software Design Specifications

for

Senior Citizen Digital Services App- Version 1.0

Prepared by:

Abhisha: Team Lead and System Architect

Sahithi: UI/UX and Accessibility Engineer

Meghana: Authentication and User Management Engineer

Sasya: Reminder and Notification Engineer

Jashitha: Emergency and Safety Features Engineer

Thrishagjna: Services Integration Engineer

Srinija: Testing, Quality and Deployment Engineer

Document Information

Title: Senior Citizens Digital Services App	Document Version No: 1.0
Project Manager: Sahithi Pinnamaraju	
Prepared By: Sahithi, Meghana, Abhisha, Sasya, Thrishagjna, Jashitha, Srinija	Document Version Date: 4/4/26
	Preparation Date: 4/4/26

Version History

Ver. No.	Ver. Date	Revised By	Description	Filename

Table of Contents

1 INTRODUCTION	4
1.1 PURPOSE	4
1.2 SCOPE	4
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS	4
1.4 REFERENCES	4
2 USE CASE VIEW	4
3 DESIGN OVERVIEW	4
3.1 DESIGN GOALS AND CONSTRAINTS	5
3.2 DESIGN ASSUMPTIONS	5
3.3 SIGNIFICANT DESIGN PACKAGES	5
3.4 DEPENDENT EXTERNAL INTERFACES	5
3.5 IMPLEMENTED APPLICATION EXTERNAL INTERFACES	5
4 LOGICAL VIEW	5
4.1 DESIGN MODEL	6
4.2 USE CASE REALIZATION	6
5 DATA VIEW	6
5.1 DOMAIN MODEL	6
5.2 DATA MODEL (PERSISTENT DATA VIEW).....	6
5.2.1 <i>Data Dictionary</i>	6
6 EXCEPTION HANDLING	6
7 CONFIGURABLE PARAMETERS	6
8 QUALITY OF SERVICE	7
8.1 AVAILABILITY.....	7
8.2 SECURITY AND AUTHORIZATION	7
8.3 LOAD AND PERFORMANCE IMPLICATIONS	7
8.4 MONITORING AND CONTROL	7

1 Introduction

This document explains the design of the Senior Citizen Digital Services App. It provides an overview of how the system works, what features it has, and how its different parts are connected. This document is mainly useful for understanding the application's structure and development.

1.1 Purpose

The purpose of this document is to describe how the Senior Citizen Digital Services App is designed. It explains the main features of the system and how they are implemented. This document can be used by developers and others to understand how the system works.

1.2 Scope

This document covers the design of the Senior Citizen Digital Services App. The app is mainly made to help senior citizens with their daily needs, like setting reminders, getting notifications, and using emergency features. It also focuses on making the app easy to use and accessible.

1.3 Definitions, Acronyms, and Abbreviations

- SDS – Software Design Specification
- UI/UX – User Interface / User Experience
- API – Application Programming Interface
- DB – Database
- Auth – Authentication

1.4 References

- IEEE guidelines for Software Design
- Class notes and materials
- Online sources used for understanding system design

2 Use Case View

This section explains how users will interact with the app and what actions they can perform. It includes the main features that users will use in the system.

User Registration

- User enters personal details like name, age, and contact information
- System stores the data in the database
- Account is successfully created

User Login

- User enters username and password
- System checks the credentials
- If correct, the user is logged into the app

Set Reminder

- User enters details of medicine or task
- User sets date and time for reminder
- System saves the reminder
- Notification is sent at the given time

Emergency Feature

- User selects emergency option
- System sends an alert message
- Message is sent to saved emergency contacts

3 Design Overview

The design of the Senior Citizen Digital Services App provides an overall view of how the system is structured. It follows a high-level modular approach where different components like the user interface, authentication, reminder system, notification system, and emergency features are connected. The design is based on system requirements and ensures that all modules work together efficiently while keeping the system simple, usable, and easy to manage.

3.1 Design Goals and Constraints

- The main goal of the system is to make the app simple and easy to use for senior citizens
- The system should be reliable and provide correct reminders and notifications
- The design should focus on accessibility, like clear text and easy navigation
- Constraints include:
 - Limited time for development
 - Basic tools and technologies used
 - Need to keep the system simple and not too complex

3.2 Design Assumptions

- It is assumed that users have basic knowledge of using a mobile phone
- An internet connection is available for notifications and services
- Users will enter correct information while using the app
- The system will run on standard mobile devices

3.3 Significant Design Packages

The system is divided into different main parts:

- User Interface module
- Authentication module
- Reminder module
- Notification module
- Emergency services module

These modules are connected and work together to form the complete system.

3.4 Dependent External Interfaces

The table below lists the public interfaces this design requires from other modules or applications.

External Application and Interface Name	Module Using the Interface	Functionality/Description
Notification Service (SMS/Push)	Reminder Module	Used to send reminders and alerts to users
Authentication Service	Login Module	Used to verify user credentials
Contact/Phone Services	Emergency Module	Used to send emergency alerts to contacts

3.5 Implemented Application External Interfaces (and SOA web services)

The table below lists the implementation of public interfaces that this design makes available for other applications.

Interface Name	Module Implementing the Interface	Functionality/Description
Reminder Interface	Reminder Module	Handles creating and managing reminders
Notification Interface	Notification Module	Sends alerts and notifications to users
Emergency Interface	Emergency Module	Sends emergency messages to saved contacts

4 Logical View

The logical view describes the detailed design of the system using different layers.

- The top layer is the **User Interface layer**, where users interact with the app to perform actions like login, setting reminders, and using emergency features
- The next layer is the **Application layer**, which handles the main logic of the system, such as processing user input, managing reminders, and sending notifications
- Below that is the **Service layer**, which manages communication between different modules and ensures they work together properly
- The bottom layer is the **Database layer**, where all user data, reminders, and other information are stored

These layers work together to complete the required functions of the system. Each module collaborates with other modules to perform tasks like registration, login, reminders, and emergency alerts. The system design can be further divided into smaller parts if needed, so that each function or method can be clearly explained in a simple way.

4.1 Design Model

The design model explains the system from a class design perspective. The system is divided into modules, and each module contains important classes.

Main classes in the system include:

- **User Class**
 - Stores user details like name, age, and contact
 - Handles user-related actions
- **Authentication Class**
 - Manages login and verification
 - Checks username and password
- **Reminder Class**
 - Stores reminder details
 - Handles scheduling of reminders
- **Notification Class**
 - Sends alerts and notifications to users
- **Emergency Class**
 - Handles emergency alerts and sends messages to contacts

Each class has its own responsibility and works with other classes. For example, the Reminder class works with the Notification class to send alerts. These classes together form the complete system design.

4.2 Use Case Realisation

This section explains how the use cases are implemented in the system.

- **For User Login:**
 - User enters details in UI
 - Authentication module verifies the data
 - Access is granted
- **For Set Reminder:**
 - User enters reminder details
 - Reminder module processes the input
 - Notification module sends alert at the correct time
- **For Emergency Feature:**
 - User activates emergency option
 - Emergency module sends alert to contacts

At a higher level, modules interact with each other to complete these actions. At a lower level, different classes inside each module work together to perform specific tasks.

5 Data View

This section describes how data is stored and managed in the system. It explains the structure of data and how different data elements are related.

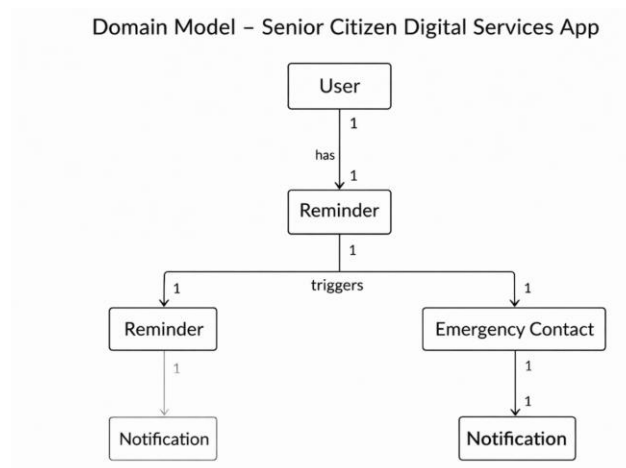
5.1 Domain Model

The domain model represents the main data entities and their relationships.

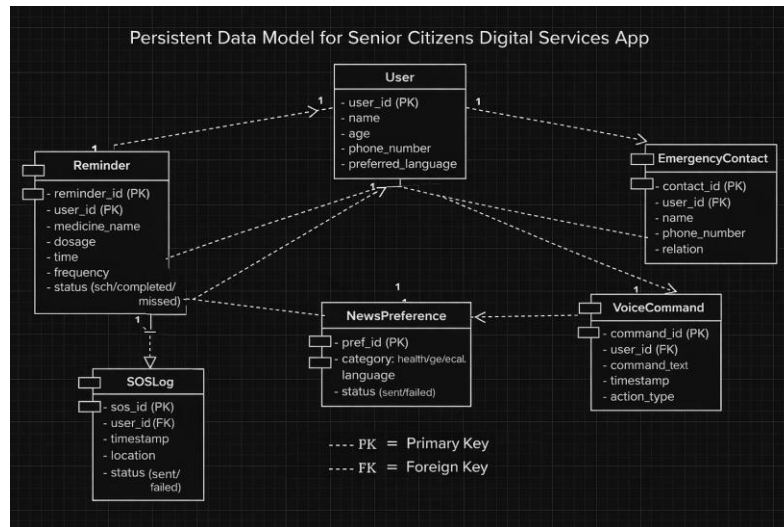
Main entities include:

- User
- Reminder
- Notification
- Emergency Contact

A user can have multiple reminders and emergency contacts. These entities are connected and represent the system data.



5.2 Data Model (persistent data view)



5.2.1 Data Dictionary

- **User Table** → stores user details
- **Reminder Table** → stores reminder information
- **Notification Table** → stores notification data
- **Emergency Table** → stores emergency contact details

6 Exception Handling

The system handles errors properly to avoid crashes.

- If login details are wrong, an error message is shown
- If reminder input is missing, the user is asked to enter details
- If notification fails, the system retries or shows an error
- All errors are handled, and messages are shown clearly to the user

7 Configurable Parameters

This table describes the simple configurable parameters (name/value pairs).

Configuration Parameter Name	Definition and Usage	Dynamic?
Reminder Time	Time set by the user for reminders:	Yes
Emergency Contacts	List of saved contacts	Yes

8 Quality of Service

This section describes how the system maintains availability, security, performance, and monitoring during its usage.

8.1 Availability

The application is designed to be available for users most of the time. It should work properly whenever users need to set reminders or use emergency features.

- The system supports continuous usage with minimal downtime
- Regular updates or maintenance may require short downtime
- Data loading and updates are handled in a way that does not affect users much

8.2 Security and Authorisation

The system ensures that user data is secure, and only authorised users can access the application.

- Users must log in using their username and password
- Authentication is used to verify user identity
- Authorisation ensures users can only access their own data
- Sensitive data like contact details is protected
- User access can be managed through the system

8.3 Load and Performance Implications

The system is designed to handle normal user load efficiently.

- The app can handle multiple users using the system at the same time
- Reminder and notification processing is done quickly
- Data is stored efficiently to avoid delays
- Performance is maintained even when the number of users increases

8.4 Monitoring and Control

The system includes monitoring features to track its performance and usage.

- The system monitors reminder and notification processes
- Errors and failures are tracked and handled
- Basic logs are maintained for system activities
- The system ensures smooth functioning by controlling different processes